EP7. Dynamic programming for the knapsack problem

Lidivine Kengne | IE 5311-001 (D01), Homework 5

1. The algorithm. Items $i \in \{1, \dots, n\}$ carry values v_i and integer weights w_i; the capacity W is an integer. The 0–1 knapsack problem is $\max \sum_i v_i y_i$ subject to $\sum_i w_i y_i \le W$, $y \in \{0,1\}^n$. Let $f(i, c)$ be the best value obtainable from the first i items under capacity c. Then $f(0, c) = 0$ and

$$f(i,c) \;=\; \begin{cases} f(i-1,c), & w_i > c, \\ \max\big\{\, f(i-1,c),\; v_i + f(i-1,\, c-w_i) \,\big\}, & w_i \le c, \end{cases}$$

the branches being *leave* and *take*. It is valid because a partial solution affects the rest only through the capacity it leaves free, so (i, c) is a sufficient state. On the Problem 2 pricing instance — $a = (2, 2, 5, 6, 2, 5, 5)$, $b = 12$, values $\pi_i - c_{i1} = (21.7, -14.0, -1.3, -5.7, -0.3, 7.3, 7.7)$ — the table is 8×13 and its rows 2 to 5 reproduce row 1 exactly, because those four items carry negative value, so *leave* always wins and the table shows directly that no optimum takes them. Backtracking reads the solution off it: item i is taken exactly when $f(i, c) \ne f(i-1, c)$, giving $y = (1, 0, 0, 0, 0, 1, 1)$ of weight $2 + 5 + 5 = 12$, filling the knapsack exactly at value $f(7, 12) = 36.7$.

2. Complexity, derived. The state space is $\{0, \dots, n\} \times \{0, \dots, W\}$, of size $(n+1)(W+1)$. Each state is evaluated once, and its evaluation is one comparison between two entries already computed, hence $\Theta(1)$ work. Total time is therefore $\Theta(nW)$ and storage $\Theta(nW)$, the latter reducible to $\Theta(W)$ by keeping one row and sweeping c downward so item i is not reused; backtracking adds $\Theta(n)$. Compare that with the *input length*: encoding the instance takes $\Theta\big(n(\log v_{\max} + \log w_{\max}) + \log W\big)$ bits, so W contributes $\log W$ bits but a factor W to the running time. The algorithm is thus exponential in the size of its input, $\Theta(nW) = \Theta(n 2^{\log W})$, which is what *pseudo-polynomial* names, and it contradicts nothing: knapsack is NP-hard. Against the $\Theta(2^n)$ of enumeration the gain is real all the same — above, 104 cells replace $2^7 = 128$ subsets.

3. When to prefer it to an IP solver. Four conditions favour the table. (i) Weights are integers and W is modest, so nW stays small. (ii) The same structure is solved repeatedly with only the objective changing, so the DP rebuilds nothing and runs a fixed number of operations where a solver pays model construction, presolve and a search on every call. This is exactly the Problem 2 pricing subproblem: for agent j the least reduced cost column solves $\min_y \sum_i (c_{ij} - \pi_i) y_i - \mu_j$ subject to $\sum_i a_i y_i \le b$, that is $-\max_y \sum_i (\pi_i - c_{ij}) y_i - \mu_j$, a knapsack whose weights a_i and capacity b never change while the values $\pi_i - c_{ij}$ change every iteration. Negative values need no special treatment, since *leave* is always available. (iii) Integer values make the recursion exact in integer arithmetic, with no tolerance and no risk of a wrong incumbent. (iv) Runtime is deterministic and known in advance, where a search tree is not. Scaling the weights and capacity by a common factor leaves the solution unchanged but grows W, which prices (i) directly (`ep7_knapsack_dp.py`):

scale	W	cells	DP (ms)	Gurobi (ms)
1	12	104	0.03	0.22
10	120	968	0.03	0.19
10^2	1 200	9 608	0.03	0.20
10^3	12 000	96 008	0.15	0.21
10^4	120 000	960 008	2.69	0.25

The DP is flat while the table fits in cache, then linear in W as $\Theta(nW)$ predicts, while the solver barely moves; the crossover is near $W \approx 2 \times 10^4$ for $n = 7$. A solver also wins once weights are fractional, since the table does not exist, and once side constraints appear, since extra capacities multiply the state space while a solver absorbs extra rows.

4. Verification. `ep7_knapsack_dp.py` substitutes the recursion for the Gurobi pricer in `p2_colgen.py` and re-runs column generation. Over the 39 pricing calls made, the dynamic program, the Gurobi binary program and enumeration of all 128 subsets returned the same reduced cost every time, and the run still reached 61.000000. Pricing cost 0.041 ms per call by table against 0.549 ms by solver, a factor of 13.5: the DP vectorised over capacities, the solver timed as column generation actually calls it, model construction included.